

Laborator 1

STRUCTURI ARITMETICE DE ADUNARE ȘI SCĂDERE

În acest laborator sunt studiate principalele componente aritmetice de adunare și scădere utilizate în unitățile de calcul.

1. Sumatorul complet pe un bit

În unitățile de calcul moderne sunt folosite circuite de adunare și scădere implementate paralel. La baza oricărei structuri de adunare se află sumatorul complet pe un bit. Construcția acestuia începe de la construirea tabelului de adevăr pentru o operație de adunare.

Sumatorul complet pe un bit este conceput pentru adunarea a două numere pe câte un bit fiecare returnând drept rezultat un bit de sumă și unul de transport. Acesta are intrări, două intrări fiind cele două numere iar cea de-a treia fiind transportul pe intrare conform figurii de mai jos:

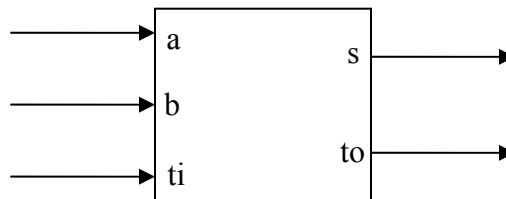


Fig. 1. Schema bloc a sumatorului complet pe un bit

Pentru adunarea a două numere pe un bit fiecare ar fi fost suficient numai intrarea a și b. Totuși în unele situații Pentru ca acest modul să se poată fi folosit în sumatoare extinse pe mai mulți biți poate fi necesar ca să existe în plus, ca intrare, și eventualul transport rezultat dintr-un proces anterior.

Pentru proiectarea sumatorului de mai sus se folosește urmatorul tabel de adevăr:

ti	a	b	s	to
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

O soluție de implementare a circuitului folosind și porți XOR (disponibile în structurile logice actuale) ar fi următoarea:

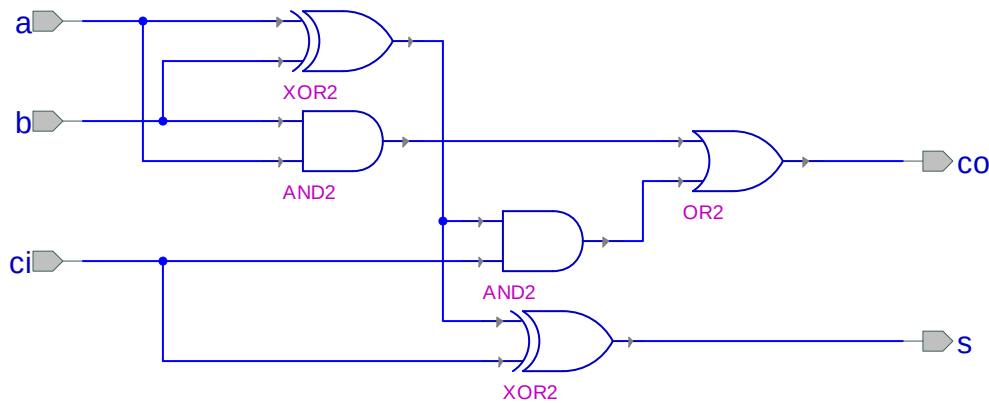


Fig. 2. Schema internă a sumatorului complet pe un bit

În practică, pentru realizarea circuitelor de adunare a doi operanzi pe mai mult de un bit, se folosesc mai multe asemenea blocuri interconectate între ele. În continuare vor fi prezentate trei asemenea tehnici pentru realizarea circuitelor de adunare luându-se drept exemplu practic adunarea a două numere pe patru biți fiecare..

2. Sumatorul cu transporturi parțiale (ripple carry)

Este cea mai simplă posibilitate de realizare a unui circuit de adunare folosind blocurile prezentate la paragraful anterior. Această metodă își are punctul de plecare de la modul în care noi adunăm două numere. De exemplu pentru două numere pe patru cifre fiecare: $a_3a_2a_1a_0$ și $b_3b_2b_1b_0$ dacă dorim să facem suma între ele adunăm în prima fază cifrele cu semnificația cea mai redusă a_0 și b_0 scrie rezultatul (s_0) și apoi continuăm adunând cifrele a_1 , b_1 și eventualul transport care a apărut la adunarea anterioară ș.a.m.d.

Urmărind această procedură se poate descrie un sumator a două numere pe patru biți astfel:

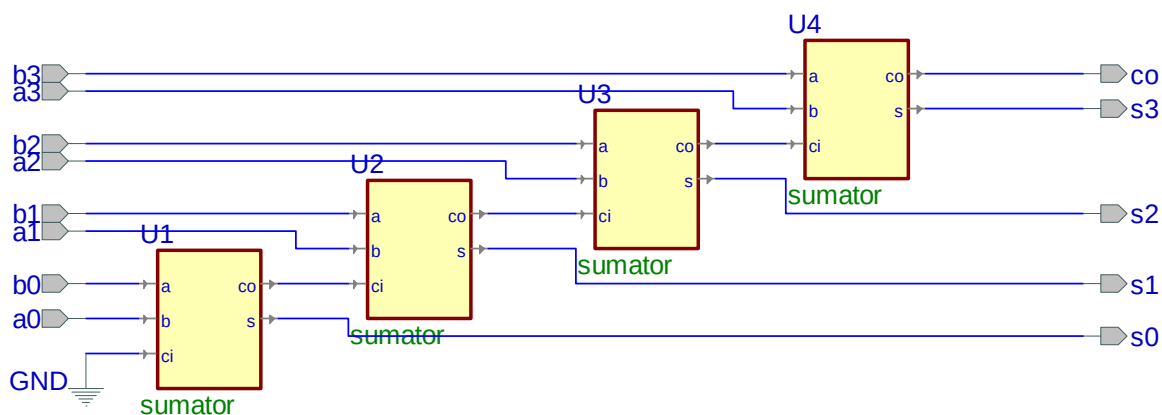


Fig. 3. Circuit adunare a două numere pe 4 biți folosind transporturi parțiale

Cele două numere sunt reprezentate în figura 3 sub forma $a_3a_2a_1a_0$ respectiv $b_3b_2b_1b_0$. Rezultatul adunării este un număr, de asemenea pe 4 biți: $s_3s_2s_1s_0$ și un bit de transport co. Cum era firesc, transportul de intrare de la primul sumator complet pe un bit este conectat la masă (0).

Circuitul, deși cel mai ușor de implementat, are un dezavantaj: timpul de răspuns (timpul scurs din momentul în care s-au aplicat pe intrare numerele până când s-a obținut la ieșire rezultatul corect) crește o dată cu creșterea lungimii operanzilor.

Dacă considerăm t_p timpul de răspuns al unei porți logice atunci pentru un sumator complet pe un bit avem un timp de răspuns egal cu $3t_p$. Prin modul în care sunt conectate blocurile în figura 3 reiese că rezultatul nu va fi corect decât după ce transportul „s-a propagat” prin toate cele patru module. Deci timpul de răspuns al circuitului va fi:

$T_p = 4 \cdot 3 t_p = 12 t_p$. Valoarea acestuia crește substanțial cu lungimea operanzilor.

Pentru un sumator a două numere pe 32 de biți de exemplu valoarea acestuia ar fi de $96 t_p$.

3. Sumatorul cu transport anticipat (carry look ahead)

Din modelul de mai sus s-a observat că principalul factor care duce la întârzieri în răspunsul circuitului de adunare îl reprezintă nu calculul sumei propriu-zise ci calculul transportului. Din acest motiv s-au realizat circuite care să poată calcula „anticipat” transportul și să-l genereze în același timp cu suma.

Ca acest lucru să fie posibil, pornind de la tabelul de adevăr de mai sus se obține următoarea funcție logică pentru obținerea transportului la ieșire la sumatorul complet pe un bit:

$c_o = (a \cdot c_i) + (b \cdot c_i) + (a \oplus b) \cdot c_i$. (în continuare se va folosi pentru reprezentarea produsului logic (SI) și pentru reprezentarea sumei logice (SAU) formele lor aritmetice. De exemplu pentru „a SI b” se va scrie „ $a \cdot b$ ” iar pentru „a SAU b” se va scrie „ $a + b$ ”). Expresia se poate reduce la (dacă $(a \oplus b) \cdot c_i$ este adevărată atunci este adevărată cel puțin una din celelalte expresii):

$$c_o = (a \cdot c_i) + (b \cdot c_i) + (a \oplus b).$$

Relația poate fi extinsă la sumatorul pe 4 biți prezentat în paragraful anterior astfel:

$$c_1 = (a_0 \cdot c_0) + (b_0 \cdot c_0) + (a_0 \oplus b_0);$$

$$c_2 = (a_1 \cdot c_1) + (b_1 \cdot c_1) + (a_1 \oplus b_1) = (a_1 \cdot a_0 \cdot c_0) + (a_1 \cdot b_0 \cdot c_0) + (a_1 \cdot a_0 \cdot b_0) + (b_1 \cdot a_0 \cdot c_0) + (b_1 \cdot b_0 \cdot c_0) + (b_1 \cdot a_0 \cdot b_0) + (a_1 \cdot b_1);$$

ș.a.m.d.

Relația se mai poate scrie și astfel:

$c_o = (a \cdot c_i) + (b \cdot c_i) + (a \oplus b) = c_i \cdot (a + b) + (a \oplus b)$. Suma logică între intrări se va nota cu p (coeficient de propagare al transportului) în timp ce produsul logic se va nota cu g (coeficient de generare al transportului).

Relația pentru transportul de ieșire de la un bloc i este:

$$c_{i+1} = (a_i \cdot b_i) + (c_i \cdot (a_i + b_i)). \text{ unde } c_i \text{ reprezintă transportul de intrare la blocul } i.$$

Deci pentru circuitul de adunare pe 4 biți vom avea:

$$c_1 = g_0 + (c_0 \cdot p_0);$$

$$c_2 = g_1 + c_1 \cdot p_1 = g_1 + (p_1 \cdot g_0) + (p_1 \cdot p_0 \cdot c_0);$$

$$c_3 = g_2 + c_2 \cdot p_2 = g_2 + (p_2 \cdot g_1) + (p_2 \cdot p_1 \cdot g_0) + (p_2 \cdot p_1 \cdot p_0 \cdot c_0);$$

$$c_4 = g_3 + c_3 \quad p_3 = g_3 + (p_3 g_2) + (p_3 p_2 g_1) + (p_3 p_2 p_1 g_0) + (p_3 p_2 p_1 p_0 c_0).$$

În figura de mai jos este prezentată schema detaliată pentru circuitul de anticipare al transportului, schemă realizată pornind de la relațiile de mai sus.

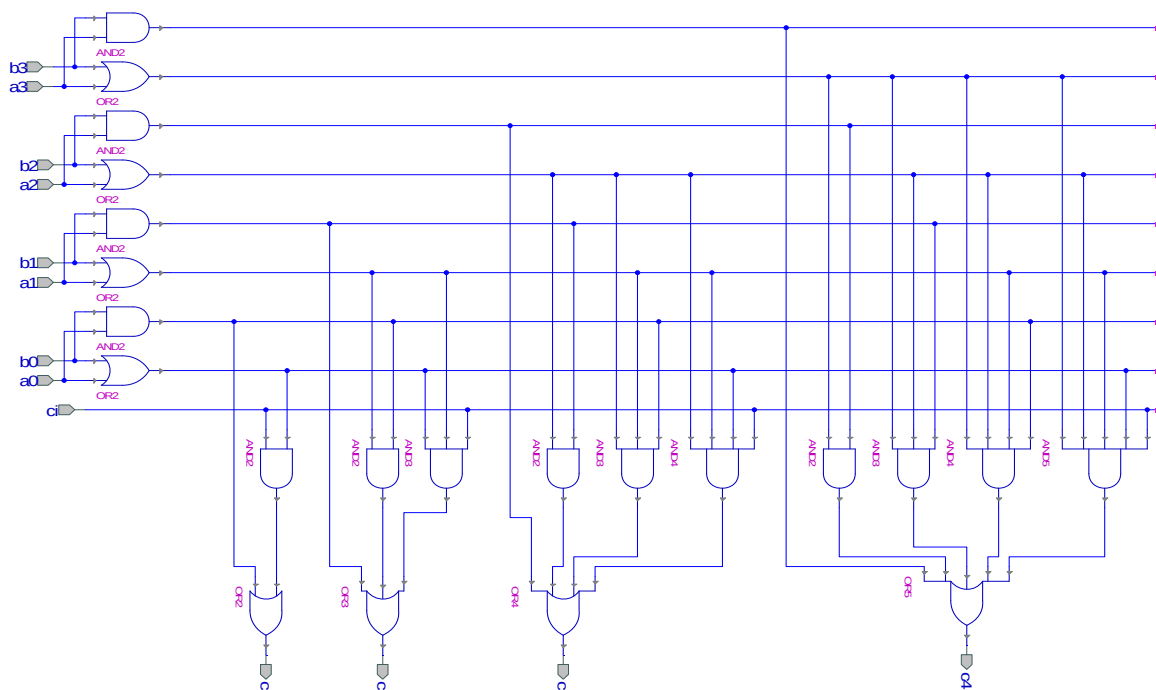


Fig. 4. Schema detaliată a circuitului de anticipare transport pe 4 biți

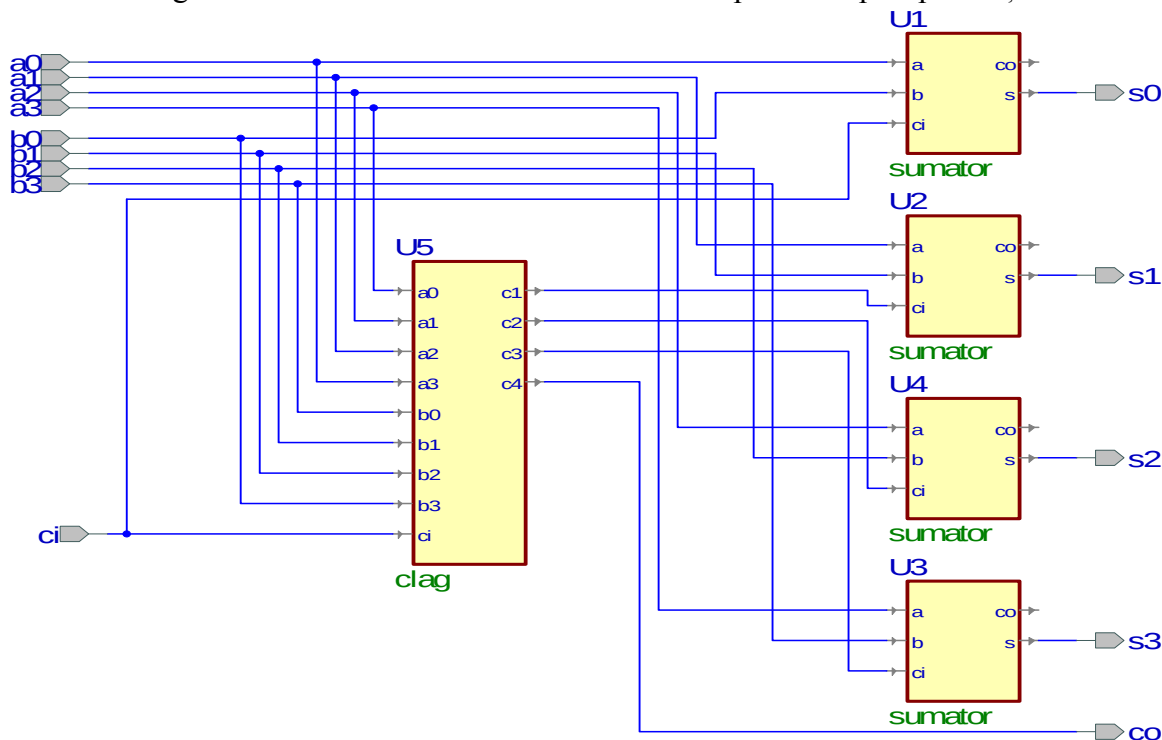


Fig. 5. Circuit adunare a două numere pe 4 biți folosind transport anticipat

În figura 5 este prezentat circuitul de adunare a două numere pe patru biți, folosind modulul de anticipare al transportului. Pentru calculul sumei au fost folosite sumatoare complete pe un bit dar se poate observa că nu mai este utilizat transportul calculat în modul.

Timpul de răspuns al acestui circuit, conform ultimelor două figuri este $T_p = 3 t_p + 3 t_p = 6 t_p$ cu t_p – timpul de răspuns al unei porți logice elementare. Acest timp se păstrează la creșterea numărului de operanzi (numărul de straturi de porți logice rămâne 3) dar apar probleme legate de fanout-ul circuitelor (numărul de circuite suportate de o intrare) și de implementarea circuitelor cu mai multe intrări. Aceste soluții sunt foarte dificile de implementat practic. În practică, aceste probleme se rezolvă prin creșterea numărului de straturi ceea ce duce deci la creșterea timpului..

4. Soluții CLAG optimizate folosite în structuri de adunare

Problemele menționate în paragraful anterior se pot rezolva prin creșterea nivelului de abstractizare. Astfel, fiecare factor de propagare și generare de la un nivel inferior pot fi însumate folosind factori de propagare și generare superiori.

Astfel:

```
c1 = g0 + (p0 c0);
```

$$c2 = g1 + (p1 \ g0) + (p1 \ p0 \ c0) = G10 + (P10 \ c0);$$

$$c3 = g2 + (p2 \ g1) + (p2 \ p1 \ g0) + (p2 \ p1 \ p0 \ c0) = g2 + (p2 \ G10) + (p2 \ P10 \ c0) = G21 + (P21 \ c0);$$

$$c_4 = g_3 + (p_3 \ g_2) + (p_3 \ p_2 \ g_1) + (p_3 \ p_2 \ p_1 \ g_0) + (p_3 \ p_2 \ p_1 \ p_0 \ c_0) = g_3 + (p_3 \ G_{21}) + (p_3 \ P_{21} \ c_0) = G_{32} + (P_{32} \ c_0).$$

În relațiile de mai sus se poate observa că se pot concepe module care pot genera cei doi factori și care apoi se vor cascadea. Structura modulului generator pentru cei doi factori este prezentată în detaliu mai jos.

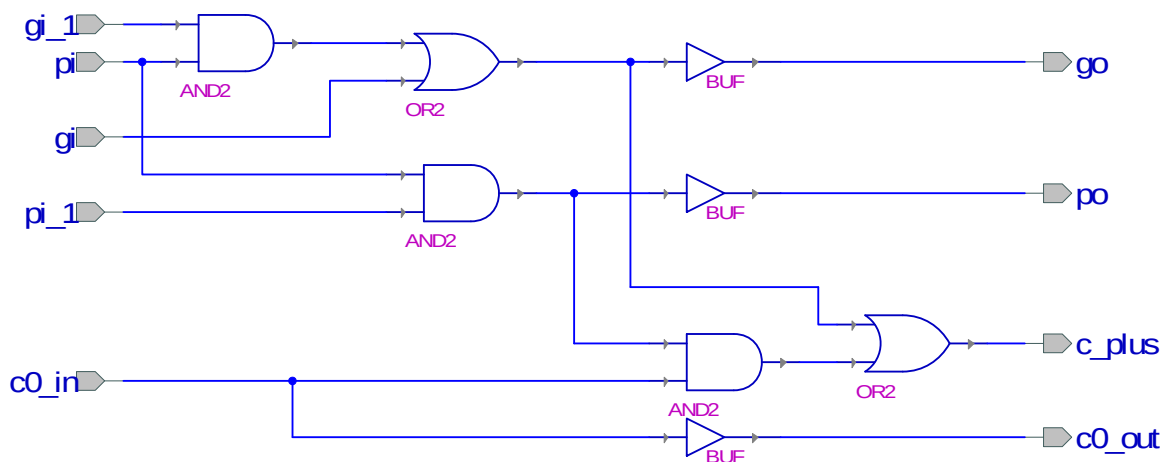


Fig. 6. Circuit calcul factori propagare și generare

Celulele de adunare vor fi de tipul următor:

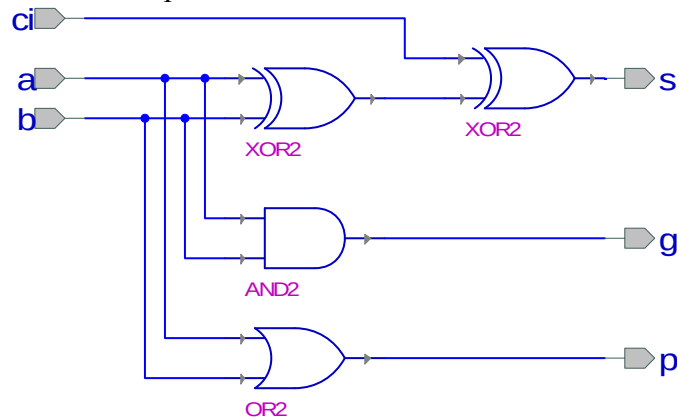


Fig. 7. Celulele de adunare și calcul g și p

Structura finală obținută folosind aceste module este:

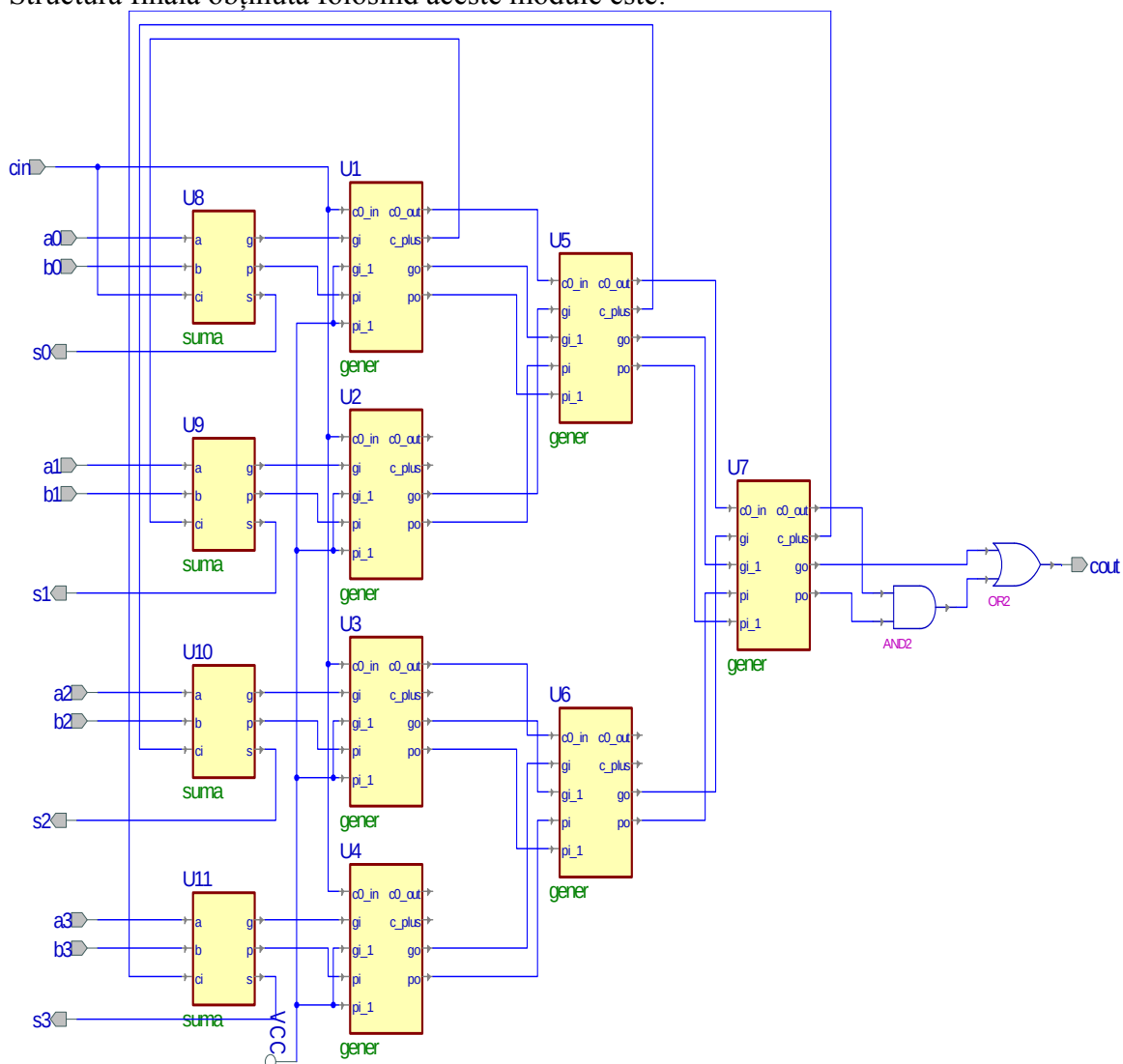


Fig. 8. Circuitul de adunare folosind module generatoare P și G

Timpul după care rezultatul este disponibil în cazul de mai sus este 12 tp însă performanțele se observă la circuitele cu mai mulți operanți. Modulele se dispun și altfel rezultând diferite tipuri de sumatoare.

5. Circuite de adunare-scădere

Scăderea se face tot folosind circuite de adunare știind că $x - y = x + (-y)$. Pentru calculul numărului negativ se pleacă de la relația $x + \bar{x} = -1$ și deci $\bar{x} + 1 = -x$. Prin urmare, se poate simplu implementa complementul numărului și aduna 1 (CC2).

Deci, pentru realizarea unui circuit de adunare scădere, cel de-al doilea operand va trebui configurat în funcție de tipul operației. Un circuit care permite configurarea celui de-al doilea operand pentru adunare și scădere are următoarea structură:

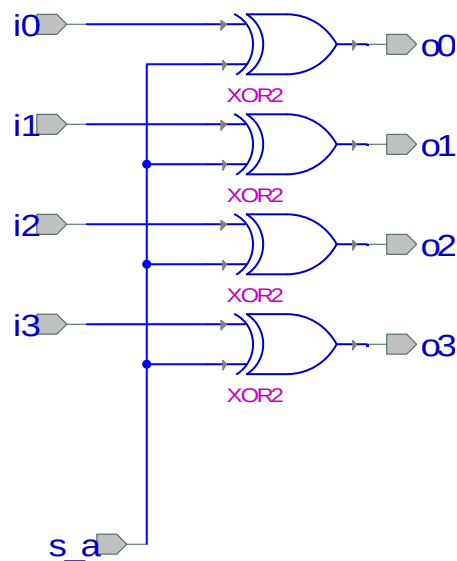


Fig. 9. Circuit inversor configurabil

Conform schemei, pentru intrarea $s_a = 1$ se realizează complementarea (inversarea) operandului în timp ce pentru $s_a = 0$ operandul rămâne neschimbat.

O problemă care apare la operarea cu numere cu semn este determinată de prezența bitului de semn în operația de adunare. De exemplu să considerăm doi operanzi pe patru biți având fiecare în plus un bit de semn (operanzi pe 5 biți cu semn). Dacă dorim să efectuăm o operație de scădere între două numere pozitive, bitul 5 reprezintă în mod corect un bit de semn, după cum se poate observa și mai jos:

$$4 - 2 = 0\ 0100(2) - 0\ 0010(2) = 0\ 0100(2) + 1\ 1110(2) = 0\ 0010(2) = 2$$

$$4 - 15 = 0\ 0100(2) - 0\ 1111(2) = 0\ 0100(2) + 1\ 0001(2) = 1\ 0101(2) = -11.$$

Pe de altă parte, în cazul unei adunări a două numere pozitive, bitul 5 va reprezenta bitul de depășire cum este prezentat în exemplul de mai jos:

$$4 + 2 = 0\ 0100(2) + 0\ 0010(2) = 0\ 0110(2) = 6$$

$$4 + 15 = 0\ 0100(2) + 0\ 1111(2) = 1\ 0011(2) = 3 \text{ (depășire) sau } 19 \text{ fără semn.}$$

Aceeași problemă apare și în cazul numerelor cu semn. De exemplu la adunarea a doua numere negative:

$$(-4) + (-5) = 1\ 1100(2) + 1\ 1011(2) = 1\ 0111(2) = -9$$

$$(-4) + (-15) = 1\ 1100(2) + 1\ 0001(2) = 0\ 1101(2) = 13 \text{ (depășire dar nesemnălizată).}$$

Pentru rezolvarea problemei generării corecte a indicatorului de depășire și de semn se vor urmări următoarele reguli:

- la adunarea a două numere cu același semn, poate apărea depășire dar nu se poate schimba semnul. Depășirea apare dacă semnul rezultatului este diferit de semnul celor două numere;
- la scăderea a două numere cu același semn nu poate apărea depășire dar semnul se schimbă, fiind calculat;
- la adunarea a două numere cu semne diferite, nu poate apărea depășire dar semnul se schimbă, fiind calculat;
- la scăderea a două numere cu semne diferite poate apărea depășire.

Deoarece operația de scădere este de fapt o operație de adunare cu unul din operanzi CC2 cele 4 reguli de mai sus se reduc de fapt la două, făcând analiza semnului operanzilor la intrarea în sumator:

- dacă cei doi operanzi au același semn rezultatul va trebui să aibă semnul operanzilor, în caz contrar înseamnă că a apărut o depășire;
- dacă cei doi operanzi au semne diferite, rezultatul va avea semnul calculat, nu poate apărea depășire.

Pe baza celor spuse mai sus se poate construi un tabel de adevăr pentru determinarea circuitului de semn și de depășire:

Semn op. 1	Semn op. 2	Semn calculat	Semn rezultat	Depășire
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	0
1	1	0	1	1
1	1	1	1	0

În figura de mai jos este prezentat un circuit care poate genera semnul și indicatorul de depășire, conform tabelului de mai sus:

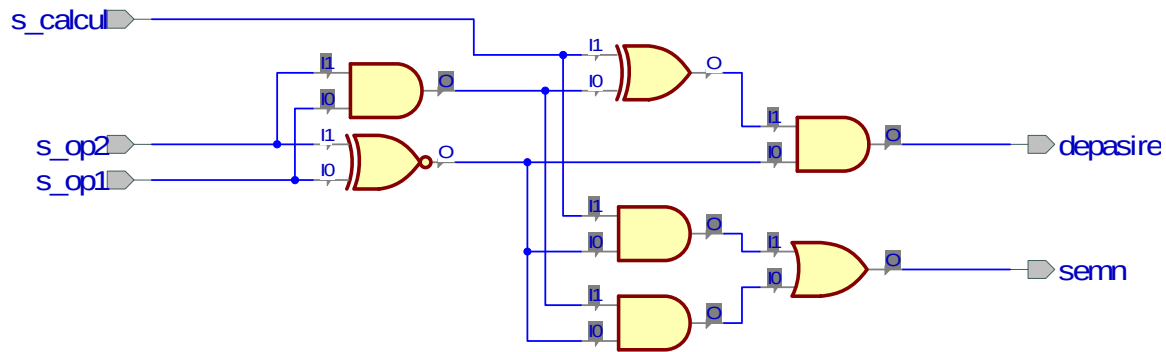


Fig. 10. Circuit calcul semn și depășire

Desfășurarea lucrării:

1. Folosind editorul schematic se va simula comportamentul următoarelor circuite:
 - sumatorul complet pe un bit;
 - circuit adunare a două numere pe patru biți cu transporturi parțiale;
 - circuit adunare a două numere pe patru biți cu transporturi anticipate;
 - circuitul calcul factor de propagare și generare precum și celula de adunare și caclul g,p;
 - circuitul de adunare a două numere pe patru biți folosind structurile CLAG optimizate.

Pentru construirea schemelor se vor folosi figurile din platforma. În toate situațiile se vor observa timpii de răspuns (în funcție de numărul de rezultate intermediare care sunt obținute până la rezultatul corect pe ieșire).

Se vor descrie și simula întâi circuitele componente apoi acestea vor fi folosite

2. Se va realiza și se va implementa pe machetă un sumator cu transporturi parțiale a două numere pe doi biți fiecare. Pentru implementare, modulul sumator complet pe un bit descris schematic, conform figurii 2 va fi înlocuit cu unul descris în cod VHDL:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
```

```
entity sumator2 is
  port(
    a : in STD_LOGIC;
    b : in STD_LOGIC;
    ci : in STD_LOGIC;
    co : out STD_LOGIC;
    s : out STD_LOGIC
  );
end sumator2;
```

```
architecture sumator2_a of sumator2 is
  signal s1,t1,t2 : std_logic;
```

```

begin

    -- comenzi pentru descrierea circuitelor logice --
    s1 <= a xor b;
    t1 <= a and b;
    s <= s1 xor ci;
    t2 <= s1 and ci;
    co <= t1 or t2;

end sumator2_a;

```

Se va crea un fișier gol de tipul VHDL Source Code în care se va introduce codul de mai sus. După compilare acest fișier se va folosi ca o componentă în construirea sumatorului conform figurii 3.

Pentru implementare se va folosi schema de mai jos (vizualizarea se face folosind un decodor 7 segmente preluat din librerie2.vhd):

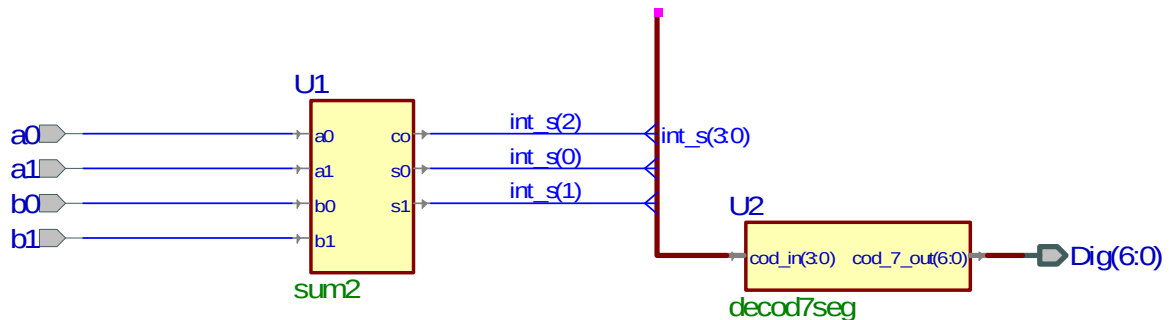


Fig. 10. Circuit testare sumator pe doi biți

Asocierea pinilor cu pinii fizici existenți pe machetă este prezentată în tabelul de mai jos:

Nume semnal	Pin machetă	Pin fizic (asociat)
A0	Sw_dip0	Y6
A1	Sw_dip 1	V6
B0	Sw_dip 2	U7
B1	Sw_dip 3	AA4
Dig(0)	Dig0_seg0	C6
Dig(1)	Dig0_seg1	B8
Dig(2)	Dig0_seg2	E7
Dig(3)	Dig0_seg3	C5
Dig(4)	Dig0_seg4	E6
Dig(5)	Dig0_seg5	B5

Dig(6)	Dig0_seg6	A4
--------	-----------	----

Întrebări și probleme. Evaluare

- Să se explice funcționarea circuitului din figura 9.
- Să se realizeze și apoi să se implementeze pe machetă un circuit de adunare scădere pe 3 biți. Se vor folosi pe post de intrări pentru operandul 1 SW1, SW2 și SW3, pentru operandul 2 SW4, SW5 și SW6 iar pentru selectarea operației SW7. Schema de operare este prezentată în figura de mai jos.

Cele două numere sunt (în ordinea importanței biților) a2a1a0 respectiv b2b1b0. Intrarea s_a comandă tipul operației („1” – scădere, „0” – adunare). Rezultatul este afișat pe un digit (7 segmente). Conversia binar bcd-7 segmente este făcută de decodorul 7 segmente (librarie2.vhd).

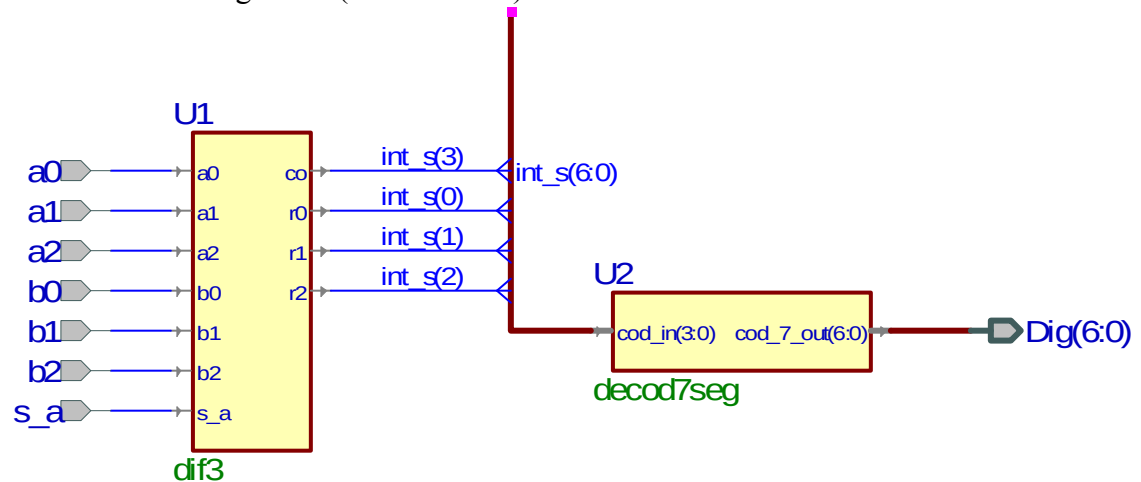


Fig. 11. Circuit testare circuit adunare-scădere pe doi biți
Pini vor fi asociați conform tabelului de mai jos:

- Să se implementeze pe machetă un circuit sumator pe doi biți folosind CLAG.

Nume semnal	Pin machetă	Pin fizic (asociat)
A0	Sw_dip0	Y6
A1	Sw_dip1	V6
A2	Sw_dip2	U7
B0	Sw_dip3	AA4
B1	Sw_dip4	AB4
B2	Sw_dip5	AA5
S A	Sw_dip6	AB5
Dig(0)	Dig0_seg0	C6
Dig(1)	Dig0_seg1	B8
Dig(2)	Dig0_seg2	E7

Dig(3)	Dig0_seg3	C5
Dig(4)	Dig0_seg4	E6
Dig(5)	Dig0_seg5	B5
Dig(6)	Dig0_seg6	A4

Grilă de evaluare:

Teorie	Punctaj
Explicare funcționare circuit figura 9	2
Schemă circuit adunare transport parțial 2 biți	2
Schemă circuit adunare CLAG optimizat 2 biți	2
Schemă circuit adunare-scădere pe 2 biți	2
Schemă circuit adunare transport parțial 8 biți	3
Schemă circuit adunare CLAG optimizat 8 biți	5
Schemă circuit adunare-scădere pe 8 biți	5
Practică	Punctaj
Implementare sumator 2 biți transport parțial	2
Implementare sumator 2 biți CLAG	2
Implementare sumator 4 biți transport anticipat	3
Implementare sumator 4 biți CLAG	5
Implementare adunare scădere 3 biți transport parțial sau CLAG	5